C++ Chapter 06 Notes

Chapter 06 — Notes

Operator precedence and associativity

Chapter introduction

This chapter builds on top of the concepts from lesson 1.9 – Introduction to literals and operators. A quick review follows:

An **operation** is a mathematical process involving zero or more input values (called **operands**) that produces a new value (called an output value). The specific operation to be performed is denoted by a construct (typically a symbol or pair of symbols) called an **operator**.

For example, as children we all learn that `2 + 3` equals `5`. In this case, the literals `2` and `3` are the operands, and the symbol `+` is the operator that tells us to apply mathematical addition on the operands to produce the new value `5`. Because there is only one operator being used here, this is straightforward.

In this chapter, we'll discuss topics related to operators, and explore many of the common operators that C++ supports.

Evaluation of compound expressions

Now, let's consider a compound expression, such as `4 + 2 * 3`. Should this be grouped as `(4 + 2) * 3` which evaluates to `18`, or `4 + (2 * 3)` which evaluates to `10`? Using normal mathematical precedence rules (which state that multiplication is resolved before addition), we know that the above expression should be grouped as `4 + (2 * 3)` to produce the value `10`. But how does the compiler know?

In order to evaluate an expression, the compiler must do two things:

- At compile time, the compiler must parse the expression and determine how operands are grouped with operators. This is done via the precedence and associativity rules, which we'll discuss momentarily.
- At compile time or runtime, the operands are evaluated and operations executed to produce a result.

Operator precedence

To assist with parsing a compound expression, all operators are assigned a level of precedence. Operators with a higher **precedence** level are grouped with operands first.

You can see in the table below that multiplication and division (precedence level 5) have a higher precedence level than addition and subtraction (precedence level 6). Thus, multiplication and division will be grouped with operands before addition and subtraction. In other words, `4 + 2 * 3` will be grouped as `4 + (2 * 3)`.

Operator associativity

Consider a compound expression like `7 - 4 - 1`. Should this be grouped as `(7 - 4) - 1` which evaluates to `2`, or `7 - (4 - 1)`, which evaluates to `4`? Since both subtraction operators have the same precedence level, the compiler can not use precedence alone to determine how this should be grouped.

If two operators with the same precedence level are adjacent to each other in an expression, the operator's **associativity** tells the compiler whether to evaluate the operators (not the operands!) from left to right or from right to left. Subtraction has precedence level 6, and the operators in precedence level 6 have an associativity of left to right. So this expression is grouped from left to right: `(7 - 4) - 1`.

Table of operator precedence and associativity

The below table is primarily meant to be a reference chart that you can refer back to in the future to resolve any precedence or associativity questions you have.

Notes:

- Precedence level 1 is the highest precedence level, and level 17 is the lowest. Operators with a higher precedence level have their operands grouped first.
- L->R means left to right associativity.
- R->L means right to left associativity.

Prec/Ass	Operator	Description	Pattern

1 L->R	`::` `::`	Global scope (unary) Namespace scope (binary)	`::name` `class_name::member_name`

2 L->R	`()` `()` `type()` `type{}` `[]` `.` `->` `++` `--` `typeid` `const_cast` `dynamic_cast` `reinterpret_cast` `static_cast` `sizeof…` `noexcept` `alignof`	Parentheses Function call Functional cast List init temporary object (C++11) Array subscript Member access from object Member access from object ptr Post-increment Post-decrement Run-time type information Cast away const Run-time type-checked cast Cast one type to another Compile-time type-checked cast Get parameter pack size Compile-time exception check Get type alignment	`(expression)`
			`function_name(arguments)`
			`type(expression)`
			`type{expression}`
			`pointer[expression]`
			`object.member_name`
			`object_pointer->member_name`
			`lvalue++` `lvalue--`
			`typeid(type) or typeid(expression)`
			`const_cast<type>(expression)`
			`dynamic_cast<type>(expression)`
			`reinterpret_cast<type>(expression)` `static_cast<type>(expression)` `sizeof…(expression)`
			`noexcept(expression)`
			`alignof(type)`

3 R->L	`+` `-` `++` `--` `!` `not` `~` `(type)` `sizeof` `co_await` `&` `*` `new` `new[]` `delete` `delete[]`	Unary plus Unary minus Pre-increment Pre-decrement Logical NOT Logical NOT Bitwise NOT C-style cast Size in bytes Await asynchronous call Address of Dereference Dynamic memory allocation Dynamic array allocation Dynamic memory deletion Dynamic array deletion	`+expression` `-expression`
			`++lvalue` `--lvalue`
			`!expression` `not expression`
			`~expression`
			`(new_type)expression`
			`sizeof(type) or sizeof(expression)` `co_await expression (C++20)` `&lvalue`
			`*expression` `new type` `new type[expression]` `delete pointer` `delete[] pointer`

4 L->R	`->*` `.*`	Member pointer selector Member object selector	`object_pointer->*pointer_to_member` `object.*pointer_to_member`

5 L->R	`*` `/` `%`	Multiplication Division Remainder	`expression * expression`
			`expression / expression`
			`expression % expression`

6 L->R	`+` `-`	Addition Subtraction	`expression + expression` `expression - expression`
7 L->R	`<<` `>>`	Bitwise shift left / Insertion Bitwise shift right / Extraction	`expression << expression` `expression >> expression`
8 L->R	`<=>`	Three-way comparison (C++20)	`expression <=> expression`

9 L->R	`<` `<=` `>` `>=`	Comparison less than
		Comparison less than or equals
		Comparison greater than
		Comparison greater than or equals
			`expression < expression`
			`expression <= expression`
			`expression > expression`
			`expression >= expression`

			`expression == expression`

10 L->R	`==` `!=`	Equality Inequality	`expression != expression`
11 L->R	`&`	Bitwise AND	`expression & expression`
12 L->R	`^`	Bitwise XOR	`expression ^ expression`
13 L->R	`|`	Bitwise OR	`expression | expression`
14 L->R	`&&` `and`	Logical AND Logical AND	`expression && expression` `expression and expression`
15 L->R	`||` `or`	Logical OR Logical OR	`expression || expression` `expression or expression`
16 R->L	`throw` `co_yield` `?:` `=` `*=` `/=` `%=` `+=` `-=` `<<=` `>>=` `&=` `|=` `^=`	Throw expression Yield expression (C++20) Conditional Assignment Multiplication assignment Division assignment Remainder assignment Addition assignment Subtraction assignment Bitwise shift left assignment Bitwise shift right assignment Bitwise AND assignment Bitwise OR assignment Bitwise XOR assignment	`throw expression` `co_yield expression` `expression ? expression : expression` `lvalue = expression` `lvalue *= expression` `lvalue /= expression` `lvalue %= expression` `lvalue += expression` `lvalue -= expression` `lvalue <<= expression` `lvalue >>= expression` `lvalue &= expression` `lvalue |= expression` `lvalue ^= expression`
17 L->R	`,`	Comma operator	`expression, expression`

You should already recognize a few of these operators, such as `+`, `-`, `*`, `/`, `()`, and `sizeof`. However, unless you have experience with another programming language, the majority of the operators in this table will probably be incomprehensible to you right now. That's expected at this point. We'll cover many of them in this chapter, and the rest will be introduced as there is a need for them.

> **Q: Where is the exponent operator?**
>
> C++ doesn't include an operator to do exponentiation (`operator^` has a different function in C++). We discuss exponentiation more in lesson 6.3 – Remainder and Exponentiation.

Note that `operator<<` handles both bitwise left shift and insertion, and `operator>>` handles both bitwise right shift and extraction. The compiler can determine which operation to perform based on the types of the operands.

Parenthesization

Due to the precedence rules, `4 + 2 * 3` will be grouped as `4 + (2 * 3)`. But what if we actually meant `(4 + 2) * 3`? Just like in normal mathematics, in C++ we can explicitly use parentheses to set the grouping of operands as we desire. This works because parentheses have one of the highest precedence levels, so parentheses generally evaluate before whatever is inside them.

Use parenthesis to make compound expressions easier to understand

Now consider an expression like `x && y || z`. Does this evaluate as `(x && y) || z` or `x && (y || z)`? You could look up in the table and see that `&&` takes precedence over `||`. But there are so many operators and precedence levels that it's hard to remember them all. And you don't want to have to look up operators all the time to understand how a compound expression evaluates.

In order to reduce mistakes and make your code easier to understand without referencing a precedence table, it's a good idea to parenthesize any non-trivial compound expression, so it's clear what your intent is.

> **Best practice**

> Use parentheses to make it clear how a non-trivial compound expression should evaluate (even if they are technically unnecessary).

A good rule of thumb is: Parenthesize everything, except addition, subtraction, multiplication, and division.

There is one additional exception to the above best practice: Expressions that have a single assignment operator (and no comma operator) do not need to have the right operand of the assignment wrapped in parenthesis.

For example:

```
x = (y + z + w);   // instead of this
x = y + z + w;     // it's okay to do this

x = ((y || z) && w); // instead of this
x = (y || z) && w;   // it's okay to do this

x = (y *= z); // expressions with multiple assignments still benefit from parenthesis
```

The assignment operators have the second lowest precedence (only the comma operator is lower, and it's rarely used). Therefore, so long as there is only one assignment (and no commas), we know the right operand will fully evaluate before the assignment.

> **Best practice**
>
> Expressions with a single assignment operator do not need to have the right operand of the assignment wrapped in parenthesis.

Value computation of operations

The C++ standard uses the term **value computation** to mean the execution of operators in an expression to produce a value. The precedence and association rules determine the order in which value computation happens.

For example, given the expression `4 + 2 * 3`, due to the precedence rules this groups as `4 + (2 * 3)`. The value computation for `(2 * 3)` must happen first, so that the value computation for `4 + 6` can be completed.

Evaluation of operands

The C++ standard (mostly) uses the term **evaluation** to refer to the evaluation of operands (not the evaluation of operators or expressions!). For example, given expression `a + b`, `a` will be evaluated to produce some value, and `b` will be evaluated to produce some value. These values can be then used as operands to `operator+` for value computation.

> **Nomenclature**
>
> Informally, we typically use the term "evaluates" to mean the evaluation of an entire expression (value computation), not just the operands of an expression.

The order of evaluation of operands (including function arguments) is mostly unspecified

In most cases, the order of evaluation for operands and function arguments is unspecified, meaning they may be evaluated in any order.

Consider the following expression:

```
a * b + c * d
```

We know from the precedence and associativity rules above that this expression will be grouped as if we had typed:

```
(a * b) + (c * d)
```

If `a` is `1`, `b` is `2`, `c` is `3`, and `d` is `4`, this expression will always compute the value `14`.

However, the precedence and associativity rules only tell us how operators and operands are grouped and the order in which value

computation will occur. They do not tell us the order in which the operands or subexpressions are evaluated. The compiler is free to evaluate operands `a`, `b`, `c`, or `d` in any order. The compiler is also free to calculate `a * b` or `c * d` first.

For most expressions, this is irrelevant. In our sample expression above, it doesn't matter in which order variables `a`, `b`, `c`, or `d` are evaluated for their values: the value calculated will always be `14`. There is no ambiguity here.

But it is possible to write expressions where the order of evaluation does matter. Consider this program, which contains a mistake often made by new C++ programmers:

```
#include <iostream>

int getValue()
{
    std::cout << "Enter an integer: ";

    int x{};
    std::cin >> x;
    return x;
}

void printCalculation(int x, int y, int z)
{
    std::cout << x + (y * z);
}

int main()
{
    printCalculation(getValue(), getValue(), getValue()); // this line is ambiguous

    return 0;
}
```

If you run this program and enter the inputs `1`, `2`, and `3`, you might assume that this program would calculate `1 + (2 * 3)` and print `7`. But that is making the assumption that the arguments to `printCalculation()` will evaluate in left-to-right order (so parameter `x` gets value `1`, `y` gets value `2`, and `z` gets value `3`). If instead, the arguments evaluate in right-to-left order (so parameter `z` gets value `1`, `y` gets value `2`, and `x` gets value `3`), then the program will print `5` instead.

> **Tip**
>
> The Clang compiler evaluates arguments in left-to-right order. The GCC compiler evaluates arguments in right-to-left order.
>
> If you'd like to see this behavior for yourself, you can do so on Wandbox. Paste in the above program, enter `1 2 3` in the *Stdin* tab, select GCC or Clang, and then compile the program. The output will appear at the bottom of the page (you may have to scroll down to see it). You will note that the output for GCC and Clang differs!

The above program can be made unambiguous by making each function call to `getValue()` a separate statement:

```
#include <iostream>

int getValue()
{
    std::cout << "Enter an integer: ";

    int x{};
    std::cin >> x;
    return x;
}

void printCalculation(int x, int y, int z)
{
    std::cout << x + (y * z);
}

int main()
{
    int a{ getValue() }; // will execute first
    int b{ getValue() }; // will execute second
    int c{ getValue() }; // will execute third

    printCalculation(a, b, c); // this line is now unambiguous

    return 0;
}
```

In this version, `a` will always have value `1`, `b` will have value `2`, and `c` will have value `3`. When the arguments to `printCalculation()` are evaluated, it doesn't matter which order the argument evaluation happens in – parameter `x

6.2 — Arithmetic operators

Unary arithmetic operators

There are two unary arithmetic operators, plus (+), and minus (-). As a reminder, unary operators are operators that only take one operand.

Operator	Symbol	Form	Operation
Unary plus	+	+x	Value of x
Unary minus	-	-x	Negation of x

The **unary minus** operator returns the operand multiplied by -1. In other words, if x = 5, -x is -5.

The **unary plus** operator returns the value of the operand. In other words, +5 is 5, and +x is x. Generally you won't need to use this operator since it's redundant. It was added largely to provide symmetry with the *unary minus* operator.

For readability, both of these operators should be placed immediately preceding the operand (e.g. `-x`, not `- x`).

Do not confuse the *unary minus* operator with the *binary subtraction* operator, which uses the same symbol. For example, in the expression `x = 5 - -3;`, the first minus is the *binary subtraction* operator, and the second is the *unary minus* operator.

Binary arithmetic operators

There are 5 binary arithmetic operators. Binary operators are operators that take a left and right operand.

Operator	Symbol	Form	Operation
Addition	+	x + y	x plus y
Subtraction	-	x - y	x minus y
Multiplication	*	x * y	x multiplied by y
Division	/	x / y	x divided by y
Remainder	%	x % y	The remainder of x divided by y

The addition, subtraction, and multiplication operators work just like they do in real life, with no caveats.

Division and remainder need some additional explanation. We'll talk about division below, and remainder in the next lesson.

Integer and floating point division

It is easiest to think of the division operator as having two different "modes".

If either (or both) of the operands are floating point values, the *division operator* performs floating point division. **Floating point division** returns a floating point value, and the fraction is kept. For example, `7.0 / 4 = 1.75`, `7 / 4.0 = 1.75`, and `7.0 / 4.0 = 1.75`. As with all floating point arithmetic operations, rounding errors may occur.

If both of the operands are integers, the *division operator* performs integer division instead. **Integer division** drops any fractions and returns an integer value. For example, `7 / 4 = 1` because the fractional portion of the result is dropped. Similarly, `-7 / 4 = -1` because the fraction is dropped.

Using static_cast<> to do floating point division with integers

The above raises the question – if we have two integers, and want to divide them without losing the fraction, how would we do so?

In lesson 4.12 – Introduction to type conversion and static_cast, we showed how we could use the *static_cast<>* operator to convert a char into an integer so it would print as an integer rather than a character.

We can similarly use *static_cast<>* to convert an integer to a floating point number so that we can do *floating point division* instead of *integer division*. Consider the following code:

```
#include <iostream>

int main()
{
    constexpr int x{ 7 };
    constexpr int y{ 4 };

    std::cout << "int / int = " << x / y << '\n';
    std::cout << "double / int = " << static_cast<double>(x) / y << '\n';
    std::cout << "int / double = " << x / static_cast<double>(y) << '\n';
    std::cout << "double / double = " << static_cast<double>(x) / static_cast<double>(y) << '\n';

    return 0;
}
```

This produces the result:

```
int / int = 1
double / int = 1.75
int / double = 1.75
double / double = 1.75
```

The above illustrates that if either operand is a floating point number, the result will be floating point division, not integer division.

Dividing by 0 and 0.0

Integer division with a divisor of `0` will cause undefined behavior, as the results are mathematically undefined!

```
#include <iostream>

int main()
{
    constexpr int apples { 12 };
    std::cout << "You have " << apples << " apples. Enter how many people to divide them between: ";
    int x {};
    std::cin >> x;

    std::cout << "Each person gets " << apples / x << " whole apples.\n"; // apples and x are int, so this is intege
division

    return 0;
}
```

If you run the above program and enter `0`, your program will likely crash. Go ahead and try it, it won't harm your computer.

The result of dividing by floating point value `0.0` is implementation-defined (meaning the behavior is determined by the compiler/architecture). On architectures that support IEEE754 floating point format, the result will be `NaN` or `Inf`. On other architectures, the result will likely be undefined behavior.

> **Related content**
>
> We discuss `NaN` and `Inf` in lesson 4.8 – Floating point numbers.

You can see what your program does by running the following program and entering `0` or `0.0`:

```
#include <iostream>

int main()
{
    constexpr int apples { 12 };
    std::cout << "You have " << apples << " apples. Enter how many servings of apples you want: ";

    double d {};
    std::cin >> d;

    std::cout << "Each serving is " << apples / d << " apples.\n"; // d is double, so this is floating point divisio

    return 0;
}
```

Arithmetic assignment operators

Operator	Symbol	Form	Operation
Addition assignment	+=	x += y	Add y to x
Subtraction assignment	-=	x -= y	Subtract y from x
Multiplication assignment	*=	x *= y	Multiply x by y
Division assignment	/=	x /= y	Divide x by y
Remainder assignment	%=	x %= y	Put the remainder of x / y in x

Up to this point, when you've needed to add 4 to a variable, you've likely done the following:

```
x = x + 4; // add 4 to existing value of x
```

This works, but it's a little clunky, and takes two operators to execute (operator+, and operator=).

Because writing statements such as `x = x + 4` is so common, C++ provides five arithmetic assignment operators for convenience. Instead of writing `x = x + 4`, you can write `x += 4`. Instead of `x = x * y`, you can write `x *= y`.

Thus, the above becomes:

```
x += 4; // add 4 to existing value of x
```

Modifying and non-modifying operators

An operator that can modify the value of one of its operands is informally called a **modifying operator**. In C++, most operators are non-modifying – they just use the operands to calculate and return a value. However, two categories of built-in operators do modify their left operand (as well as return a value):

- The assignment operators, including the standard assignment operator (`=`), the arithmetic assignment operators (`+=`, `-=`, `*=`, `/=`, and `%=`), and the bitwise assignment operators (`<<=`, `>>=`, `&=`, `|=` and `^=`).
- The increment and decrement operators (`++` and `--` respectively). We discuss these in lesson 6.4 – Increment/decrement operators, and side effects.

Excluded from this list are operators `==`, `!=`, `<=`, and `>=`, as these are non-modifying relational (comparison) operators (the `=` means "is equal to"). We discuss these in lesson 6.7 – Relational operators and floating point comparisons.

> **For advanced readers**
>
> Overloaded operators can be redefined to have different behavior than built-in operators, and this may include modifying the left operand even if the built-in version does not (or vice-versa). For example, the overloaded version of `operator<<` used for output modifies its left operand (the output stream object).

6.3 — Remainder and Exponentiation

The remainder operator (`operator%`)

The **remainder operator** (also commonly called the **modulo operator** or **modulus operator**) is an operator that returns the remainder after doing an integer division. For example, 7 / 4 = 1 remainder 3. Therefore, 7 % 4 = 3. As another example, 25 / 7 = 3 remainder 4, thus 25 % 7 = 4. The remainder operator only works with integer operands.

This is most useful for testing whether a number is evenly divisible by another number (meaning that after division, there is no remainder): if *x % y* evaluates to 0, then we know that *x* is evenly divisible by *y*.

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";
    int x{};
    std::cin >> x;

    std::cout << "Enter another integer: ";
    int y{};
    std::cin >> y;

    std::cout << "The remainder is: " << x % y << '\n';

    if ((x % y) == 0)
        std::cout << x << " is evenly divisible by " << y << '\n';
    else
        std::cout << x << " is not evenly divisible by " << y << '\n';

    return 0;
}
```

Here are a couple runs of this program:

```
Enter an integer: 6
Enter another integer: 3
The remainder is: 0
6 is evenly divisible by 3
```

```
Enter an integer: 6
Enter another integer: 4
The remainder is: 2
6 is not evenly divisible by 4
```

Now let's try an example where the second number is bigger than the first:

```
Enter an integer: 2
Enter another integer: 4
The remainder is: 2
2 is not evenly divisible by 4
```

A remainder of 2 might be a little non-obvious at first, but it's simple: 2 / 4 is 0 (using integer division) remainder 2. Whenever the second number is larger than the first, the second number will divide the first 0 times, so the first number will be the remainder.

Remainder with negative numbers

The remainder operator can also work with negative operands. `x % y` always returns results with the sign of *x*.

Running the above program:

```
Enter an integer: -6
Enter another integer: 4
The remainder is: -2
-6 is not evenly divisible by 4
```

```
Enter an integer: 6
Enter another integer: -4
The remainder is: 2
6 is not evenly divisible by -4
```

In both cases, you can see the remainder takes the sign of the first operand.

Nomenclature

The C++ standard does not actually give a name to `operator%`. However, the C++20 standard does say, "the binary % operator yields the remainder from the division of the first expression by the second".

Although `operator%` is often called the "modulo" or "modulus" operator, this can be confusing, because modulo in mathematics is often defined in a way that yields a different result to what `operator%` in C++ produces when one (and only one) of the operands is negative.

For example, in mathematics:

-21 modulo 4 = 3

-21 remainder 4 = -1

For this reason, we believe the name "remainder" is a more accurate name for `operator%` than "modulo".

In cases where the first operand can be negative, one must take care to note that the remainder can also be negative. For example, you might think to write a function that returns whether a number is odd like this:

```
bool isOdd(int x)
{
    return (x % 2) == 1; // fails when x is -5
}
```

However, this will fail when x is a negative odd number, such as `-5`, because `-5 % 2` is -1, and -1 != 1.

For this reason, if you're going to compare the result of a remainder operation, it's better to compare against 0, which does not have positive/negative number issues:

```
bool isOdd(int x)
{
    return (x % 2) != 0; // could also write return (x % 2)
}
```

> **Best practice**
>
> Prefer to compare the result of the remainder operator (`operator%`) against 0 if possible.

Where's the exponent operator?

You'll note that the ^ operator (commonly used to denote exponentiation in mathematics) is a *Bitwise XOR* operation in C++ (covered in lesson O.3 – Bit manipulation with bitwise operators and bit masks). C++ does not include an exponent operator.

To do exponents in C++, `#include` the `<cmath>` header, and use the `pow()` function:

```
#include <cmath>

double x{ std::pow(3.0, 4.0) }; // 3 to the 4th power
```

Note that the parameters (and return value) of function `pow()` are of type `double`. Due to rounding errors in floating point numbers, the results of `pow()` may not be precise (even if you pass it integers or whole numbers).

If you want to do integer exponentiation, you're best off using your own function to do so. The following function implements integer exponentiation (using the non-intuitive "exponentiation by squaring" algorithm for efficiency):

```
#include <cassert> // for assert
#include <cstdint> // for std::int64_t
#include <iostream>

// note: exp must be non-negative
// note: does not perform range/overflow checking, use with caution
constexpr std::int64_t powint(std::int64_t base, int exp)
{
    assert(exp >= 0 && "powint: exp parameter has negative value");

    // Handle 0 case
    if (base == 0)
        return (exp == 0) ? 1 : 0;

    std::int64_t result{ 1 };
    while (exp > 0)
    {
        if (exp & 1)  // if exp is odd
            result *= base;
        exp /= 2;
        base *= base;
    }

    return result;
}

int main()
{
    std::cout << powint(7, 12) << '\n'; // 7 to the 12th power

    return 0;
}
```

Produces:

```
13841287201
```

Don't worry if you don't understand how this function works – you don't need to understand it in order to call it.

> **Related content**
>
> We cover asserts in lesson 9.6 – Assert and static_assert, and constexpr functions in lesson F.1 – Constexpr functions.

> **For advanced readers**
>
> The `constexpr` specifier allows a function to be evaluated at compile-time if used in a constant expression; otherwise, it behaves like a regular function and is evaluated at runtime.

> **Warning**
>
> In the vast majority of cases, integer exponentiation will overflow the integral type. This is likely why such a function wasn't included in the standard library in the first place.

Here is a safer version of the exponentiation function above that checks for overflow:

```
#include <cassert> // for assert
#include <cstdint> // for std::int64_t
#include <iostream>
#include <limits> // for std::numeric_limits

// A safer (but slower) version of powint() that checks for overflow
// note: exp must be non-negative
// Returns std::numeric_limits<std::int64_t>::max() if overflow occurs
constexpr std::int64_t powint_safe(std::int64_t base, int exp)
{
    assert(exp >= 0 && "powint_safe: exp parameter has negative value");

    // Handle 0 case
    if (base == 0)
        return (exp == 0) ? 1 : 0;

    std::int64_t result { 1 };

    // To make the range checks easier, we'll ensure base is positive
    // We'll flip the result at the end if needed
    bool negativeResult{ false };

    if (base < 0)
    {
        base = -base;
        negativeResult = (exp & 1);
    }

    while (exp > 0)
    {
        if (exp & 1) // if exp is odd
        {
            // Check if result will overflow when multiplied by base
```

```
            if (result > std::numeric_limits<std::int64_t>::max() / base)
            {
                std::cerr << "powint_safe(): result overflowed\n";
                return std::numeric_limits<std::int64_t>::max();
            }

            result *= base;
        }

        exp /= 2;

        // If we're done, get out here
        if (exp <= 0)
            break;

        // The following only needs to execute if we're going to iterate again

        // Check if base will overflow when multiplied by base
        if (base > std::numeric_limits<std::int64_t>::max() / base)
        {
            std::cerr << "powint_safe(): base overflowed\n";
            return std::numeric_limits<std::int64_t>::max();
        }

        base *= base;
    }

    if (negativeResult)
        return -result;

    return result;
}

int main()
{
    std::cout << powint_safe(7, 12) << '\n'; // 7 to the 12th power
    std::cout << powint_safe(70, 12) << '\n'; // 70 to the 12th power (will return the max 64-bit int value)

    return 0;
}
```

6.4 — Increment/decrement operators, and side effects

Incrementing and decrementing variables

Incrementing (adding 1 to) and decrementing (subtracting 1 from) a variable are both so common that they have their own operators.

Operator	Symbol	Form	Operation
Prefix increment (pre-increment)	++	++x	Increment x, then return x
Prefix decrement (pre-decrement)	—	—x	Decrement x, then return x
Postfix increment (post-increment)	++	x++	Copy x, then increment x, then return the copy
Postfix decrement (post-decrement)	—	x—	Copy x, then decrement x, then return the copy

Note that there are two versions of each operator – a prefix version (where the operator comes before the operand) and a postfix version (where the operator comes after the operand).

Prefix increment and decrement

The prefix increment/decrement operators are very straightforward. First, the operand is incremented or decremented, and then expression evaluates to the value of the operand. For example:

```
#include <iostream>

int main()
{
    int x { 5 };
    int y { ++x }; // x is incremented to 6, x is evaluated to the value 6, and 6 is assigned to y

    std::cout << x << ' ' << y << '\n';
    return 0;
}
```

This prints:

```
6 6
```

Postfix increment and decrement

The postfix increment/decrement operators are trickier. First, a copy of the operand is made. Then the operand (not the copy) is incremented or decremented. Finally, the copy (not the original) is evaluated. For example:

```
#include <iostream>

int main()
{
    int x { 5 };
    int y { x++ }; // x is incremented to 6, copy of original x is evaluated to the value 5, and 5 is assigned to y

    std::cout << x << ' ' << y << '\n';
    return 0;
}
```

This prints:

```
6 5
```

Let's examine how this line 6 works in more detail. First, a temporary copy of *x* is made that starts with the same value as *x* (5). Then the actual *x* is incremented from *5* to *6*. Then the copy of *x* (which still has value *5*) is returned and assigned to *y*. Then the temporary copy is discarded.

Consequently, *y* ends up with the value of *5* (the pre-incremented value), and *x* ends up with the value *6* (the post-incremented value).

Note that the postfix version takes a lot more steps, and thus may not be as performant as the prefix version.

More examples

Here is another example showing the difference between the prefix and postfix versions:

```
#include <iostream>

int main()
{
    int x { 5 };
    int y { 5 };
    std::cout << x << ' ' << y << '\n';
    std::cout << ++x << ' ' << --y << '\n'; // prefix
    std::cout << x << ' ' << y << '\n';
    std::cout << x++ << ' ' << y-- << '\n'; // postfix
    std::cout << x << ' ' << y << '\n';

    return 0;
}
```

This produces the output:

```
5 5
6 4
6 4
6 4
7 3
```

On the 8th line, we do a prefix increment and decrement. On this line, *x* and *y* are incremented/decremented *before* their values are sent to std::cout, so we see their updated values reflected by std::cout.

On the 10th line, we do a postfix increment and decrement. On this line, the copy of *x* and *y* (with the pre-incremented and pre-

decremented values) are what is sent to std::cout, so we don't see the increment and decrement reflected here. Those changes don't show up until the next line, when *x* and *y* are evaluated again.

When to use prefix vs postfix

In many cases, the prefix and postfix operators produce the same behavior:

```
int main()
{
    int x { 0 };
    ++x; // increments x to 1
    x++; // increments x to 2

    return 0;
}
```

In cases where code can be written to use either prefix or postfix, prefer the prefix versions, as they are generally more performant, and less likely to cause surprises.

> **Best practice**
>
> Favor the prefix versions, as they are more performant and less likely to cause surprises.
>
> Use the postfix versions when doing so produces significantly more concise or understandable code than the equivalent code written using the prefix versions.

Side effects

A function or expression is said to have a **side effect** if it has some observable effect beyond producing a return value.

Common examples of side effects include changing the value of objects, doing input or output, or updating a graphical user interface (e.g. enabling or disabling a button).

Most of the time, side effects are useful:

```
x = 5; // the assignment operator has side effect of changing value of x
++x; // operator++ has side effect of incrementing x
std::cout << x; // operator<< has side effect of modifying the state of the console
```

The assignment operator in the above example has the side effect of changing the value of *x* permanently. Even after the statement has finished executing, *x* will still have the value 5. Similarly with operator++, the value of *x* is altered even after the statement has finished evaluating. The outputting of *x* also has the side effect of modifying the state of the console, as you can now see the value of *x* printed to the console.

> **Key insight**
>
> The assignment operators, prefix operator, and postfix operator have side effects that permanently change the value of an object. Other operators (such as the arithmetic operators) return a value, and do not modify their operands.

Side effects can cause order of evaluation issues

In some cases, side effects can lead to order of evaluation problems. For example:

```
#include <iostream>

int add(int x, int y)
{
    return x + y;
}

int main()
{
    int x { 5 };
    int value{ add(x, ++x) }; // undefined behavior: is this 5 + 6, or 6 + 6?
    // It depends on what order your compiler evaluates the function arguments in

    std::cout << value << '\n'; // value could be 11 or 12, depending on how the above line evaluates!

    return 0;
}
```

The C++ standard does not define the order in which function arguments are evaluated. If the left argument is evaluated first, this becomes a call to add(5, 6), which equals 11. If the right argument is evaluated first, this becomes a call to add(6, 6), which equals 12! Note that this is only a problem because one of the arguments to function add() has a side effect.

> **As an aside…**
>
> The C++ standard intentionally does not define these things so that compilers can do whatever is most natural (and thus most performant) for a given architecture.

The sequencing of side effects

In many cases, C++ also does not specify when the side effects of operators must be applied. This can lead to undefined behavior in cases where an object with a side effect applied is used more than once in the same statement.

For example, the expression `x + ++x` is unspecified behavior. When `x` is initialized to `1`, Visual Studio and GCC evaluate this as `2 + 2`, and Clang evaluates it as `1 + 2`! This is due to differences in when the compilers apply the side effect of incrementing `x`.

Even when the C++ standard does make it clear how things should be evaluated, historically this has been an area where there have been many compiler bugs. These problems can generally *all* be avoided by ensuring that any variable that has a side-effect applied is used no more than once in a given statement.

> **Warning**
>
> C++ does not define the order of evaluation for function arguments or the operands of operators.

> **Warning**
>
> Don't use a variable that has a side effect applied to it more than once in a given statement. If you do, the result may be undefined.

One exception is for simple assignment expressions such as `x = x + y` (which is essentially equivalent to `x += y`).

6.5 — The comma operator

Operator	Symbol	Form	Operation
Comma	,	x, y	Evaluate x then y, returns value of y

The **comma operator (,)** allows you to evaluate multiple expressions wherever a single expression is allowed. The comma operator evaluates the left operand, then the right operand, and then returns the result of the right operand.

For example:

```
#include <iostream>

int main()
{
    int x{ 1 };
    int y{ 2 };

    std::cout << (++x, ++y) << '\n'; // increment x and y, evaluates to the right operand

    return 0;
}
```

First the left operand of the comma operator is evaluated, which increments *x* from *1* to *2*. Next, the right operand is evaluated, which increments *y* from *2* to *3*. The comma operator returns the result of the right operand (*3*), which is subsequently printed to the console.

Note that comma has the lowest precedence of all the operators, even lower than assignment. Because of this, the following two lines of code do different things:

```
z = (a, b); // evaluate (a, b) first to get result of b, then assign that value to variable z.
z = a, b; // evaluates as "(z = a), b", so z gets assigned the value of a, and b is evaluated and discarded.
```

This makes the comma operator somewhat dangerous to use.

In almost every case, a statement written using the comma operator would be better written as separate statements. For example, the above code could be written as:

```
#include <iostream>

int main()
{
    int x{ 1 };
    int y{ 2 };

    ++x;
    std::cout << ++y << '\n';

    return 0;
}
```

Most programmers do not use the comma operator at all, with the single exception of inside *for loops*, where its use is fairly common. We discuss for loops in future lesson 8.10 – For statements.

> **Best practice**
>
> Avoid using the comma operator, except within *for loops*.

Comma as a separator

In C++, the comma symbol is often used as a separator, and these uses do not invoke the comma operator. Some examples of separator commas:

```
void foo(int x, int y) // Separator comma used to separate parameters in function definition
{
    add(x, y); // Separator comma used to separate arguments in function call
    constexpr int z{ 3 }, w{ 5 }; // Separator comma used to separate multiple variables being defined on the same l
(don't do this)
}
```

There is no need to avoid separator commas (except when declaring multiple variables, which you should not do).

6.6 — The conditional operator

Operator	Symbol	Form	Meaning
Conditional	?:	c ? x : y	If conditional `c` is `true` then evaluate `x`, otherwise evaluate `y`

The **conditional operator** (`?:`) (also sometimes called the **arithmetic if** operator) is a ternary operator (an operator that takes 3 operands). Because it has historically been C++'s only ternary operator, it's also sometimes referred to as "the ternary operator".

The `?:` operator provides a shorthand method for doing a particular type of if-else statement.

> **Related content**
>
> We cover if-else statements in lesson 4.10 – Introduction to if statements.

To recap, an if-else statement takes the following form:

```
if (condition)
    statement1;
else
    statement2;
```

If `condition` evaluates to `true`, then `statement1` is executed, otherwise `statement2` is executed. The `else` and `statement2` are optional.

The `?:` operator takes the following form:

```
condition ? expression1 : expression2;
```

If `condition` evaluates to `true`, then `expression1` is evaluated, otherwise `expression2` is evaluated. The `:` and `expression2` are not optional.

Consider an if-else statement that looks like this:

```
if (x > y)
    max = x;
else
    max = y;
```

This can be rewritten as:

```
max = ((x > y) ? x : y);
```

In such cases, the conditional operator can help compact code without losing readability.

An example

Consider the following example:

```
#include <iostream>

int getValue()
{
    std::cout << "Enter a number: ";
    int x{};
    std::cin >> x;
    return x;
}

int main()
{
    int x { getValue() };
    int y { getValue() };
    int max { (x > y) ? x : y };
    std::cout << "The max of " << x <<" and " << y << " is " << max << ".\n";

    return 0;
}
```

First, let's enter `5` and `7` as input (so `x` is `5`, and `y` is `7`). When `max` is initialized, the expression `(5 > 7) ? 5 : 7` is evaluated. Since `5 > 7` is false, this yields `false ? 5 : 7`, which evaluates to `7`. The program prints:

```
The max of 5 and 7 is 7.
```

Now let's enter `7` and `5` as input (so `x` is `7`, and `y` is `5`). In this case, we get `(7 > 5) ? 7 : 5`, which is `true ? 7 : 5`, which evaluates to `7`. The program prints:

```
The max of 7 and 5 is 7.
```

The conditional operator evaluates as part of an expression

Since the conditional operator is evaluated as part of an expression, it can be used anywhere an expression is accepted. In cases where the operands of the conditional operator are constant expressions, the conditional operator can be used in a constant expression.

This allows the conditional operator to be used in places where statements cannot be used.

For example, when initializing a variable:

```
#include <iostream>

int main()
{
    constexpr bool inBigClassroom { false };
    constexpr int classSize { inBigClassroom ? 30 : 20 };
    std::cout << "The class size is: " << classSize << '\n';

    return 0;
}
```

There's no direct if-else replacement for this.

You might think to try something like this:

```cpp
#include <iostream>

int main()
{
    constexpr bool inBigClassroom { false };

    if (inBigClassroom)
        constexpr int classSize { 30 };
    else
        constexpr int classSize { 20 };

    std::cout << "The class size is: " << classSize << '\n'; // Compile error: classSize not defined

    return 0;
}
```

However, this won't compile, and you'll get an error message that `classSize` isn't defined. Much like how variables defined inside functions die at the end of the function, variables defined inside an if-statement or else-statement die at the end of the if-statement or else-statement. Thus, `classSize` has already been destroyed by the time we try to print it.

If you want to use an if-else, you'd have to do something like this:

```cpp
#include <iostream>

int getClassSize(bool inBigClassroom)
{
    if (inBigClassroom)
        return 30;
    else
        return 20;
}

int main()
{
    const int classSize { getClassSize(false) };
    std::cout << "The class size is: " << classSize << '\n';

    return 0;
}
```

This one works because `getClassSize(false)` is an expression, and the if-else logic is inside a function (where we can use statements). But this is a lot of extra code when we could just use the conditional operator instead.

Parenthesizing the conditional operator

Because C++ prioritizes the evaluation of most operators above the evaluation of the conditional operator, it's quite easy to write expressions using the conditional operator that don't evaluate as expected.

> **Related content**
>
> We cover the way that C++ prioritizes the evaluation of operators in future lesson 6.1 – Operator precedence and associativity.

For example:

```
#include <iostream>

int main()
{
    int x { 2 };
    int y { 1 };
    int z { 10 - x > y ? x : y };
    std::cout << z;

    return 0;
}
```

You might expect this to evaluate as `10 - (x > y ? x : y)` (which would evaluate to `8`) but it actually evaluates as `(10 - x) > y ? x : y` (which evaluates to `2`).

Here's another example that exhibits a common mistake:

```
#include <iostream>

int main()
{
    int x { 2 };
    std::cout << (x < 0) ? "negative" : "non-negative";

    return 0;
}
```

You might expect this to print `non-negative`, but it actually prints `0`.

> **Optional reading**
>
> Here's what's happening in the above example. First, `x < 0` evaluates to `false`. The partially evaluated expression is now `std::cout << false ? "negative" : "non-negative"`. Because `operator<<` has higher precedence than `operator?:`, this expression evaluates as if it were written as `(std::cout << false) ? "negative" : "non-negative"`. Thus `std::cout << false` is evaluated, which prints `0` (and returns `std::cout`).
>
> The partially evaluated expression is now `std::cout ? "negative" : "non-negative"`. Since `std::cout` is all that is remaining in the condition, the compiler will try to convert it to a `bool` so the condition can be resolved. Perhaps surprisingly, `std::cout` has a defined conversion to `bool`, which will most likely return `false`. Assuming it returns `false`, we now have `false ? "negative" : "non-negative"`, which evaluates to `"non-negative"`. So our fully evaluated statement is `"non-negative";`. An expression statement that is just a literal (in this case, a string literal) has no effect, so we're done.

To avoid such evaluation prioritization issues, the conditional operator should be parenthesized as follows:

- Parenthesize the entire conditional operation (including operands) when used in a compound expression (an expression with other operators).
- For readability, consider parenthesizing the condition if it contains any operators (other than the function call operator).

The operands of the conditional operator do not need to be parenthesized.

Let's take a look at some statements containing the conditional operator and how they should be parenthesized:

```
        return isStunned ? 0 : movesLeft;          // not used in compound expression, condition contains no operators
        int z { (x > y) ? x : y };                 // not used in compound expression, condition contains operators
        std::cout << (isAfternoon() ? "PM" : "AM"); // used in compound expression, condition contains no operators (functio
operator excluded)
        std::cout << ((x > y) ? x : y);            // used in compound expression, condition contains operators
```

> **Best practice**
>
> Parenthesize the entire conditional operation (including operands) when used in a compound expression.
>
> For readability, consider parenthesizing the condition if it contains any operators (other than the function call operator).

The type of the expressions must match or be convertible

To comply with C++'s type checking rules, one of the following must be true:

- The type of the second and third operand must match.
- The compiler must be able to find a way to convert one or both of the second and third operands to matching types. The conversion rules the compiler uses are fairly complex and may yield surprising results in some cases.

> **For advanced readers**
>
> Alternatively, one or both of the second and third operands is allowed to be a throw expression. We cover `throw` in lesson 27.2 – Basic exception handling.

For example:

```
        #include <iostream>

        int main()
        {
            std::cout << (true ? 1 : 2) << '\n';    // okay: both operands have matching type int

            std::cout << (false ? 1 : 2.2) << '\n'; // okay: int value 1 converted to double

            std::cout << (true ? -1 : 2u) << '\n';  // surprising result: -1 converted to unsigned int, result out of range

            return 0;
        }
```

Assuming 4 byte integers, the above prints:

```
1
2.2
4294967295
```

In general, it's okay to mix operands with fundamental types (excluding mixing signed and unsigned values). If either operand is not a fundamental type, it's generally best to explicitly convert one or both operands to a matching type yourself so you know exactly what you'll get.

> **Related content**
>
> The surprising case above related to mixing signed and unsigned values is due to the arithmetic conversion rules, which we cover in lesson 10.5 – Arithmetic conversions.

If the compiler can't find a way to convert the second and third operands to a matching type, a compile error will result:

```
#include <iostream>

int main()
{
    constexpr int x{ 5 };
    std::cout << ((x != 5) ? x : "x is 5"); // compile error: compiler can't find common type for constexpr int and style string literal

    return 0;
}
```

In the above example, one of the expressions is an integer, and the other is a C-style string literal. The compiler will not be able to find a matching type on its own, so a compile error will result.

In such cases, you can either do an explicit conversion, or use an if-else statement:

```
#include <iostream>
#include <string>

int main()
{
    int x{ 5 }; // intentionally non-constexpr for this example

    // We can explicitly convert the types to match
    std::cout << ((x != 5) ? std::to_string(x) : std::string{"x is 5"}) << '\n';

    // Or use an if-else statement
    if (x != 5)
        std::cout << x << '\n';
    else
        std::cout << "x is 5" << '\n';

    return 0;
}
```

For advanced readers

If `x` is constexpr, then the condition `x != 5` is a constant expression. In such cases, using `if constexpr` should be preferred over `if`, and your compiler may generate a warning indicating so (which will be promoted to an error if you are treating warnings as errors).

Since we haven't covered `if constexpr` yet (we do so in lesson 8.4 – Constexpr if statements), `x` is non-constexpr in this example to avoid the potential compiler warning.

So when should you use the conditional operator?

The conditional operator is most useful when doing one of the following:

- Initializing an object with one of two values.
- Assigning one of two values to an object.
- Passing one of two values to a function.
- Returning one of two values from a function.

- Printing one of two values.

Complicated expressions should generally avoid use of the conditional operator, as they tend to be error prone and hard to read.

> **Best practice**
>
> Prefer to avoid the conditional operator in complicated expressions.

6.7 — Relational operators and floating point comparisons

Relational operators are operators that let you compare two values. There are 6 relational operators:

Operator	Symbol	Form	Operation
Greater than	>	x > y	true if x is greater than y, false otherwise
Less than	<	x < y	true if x is less than y, false otherwise
Greater than or equals	>=	x >= y	true if x is greater than or equal to y, false otherwise
Less than or equals	<=	x <= y	true if x is less than or equal to y, false otherwise
Equality	==	x == y	true if x equals y, false otherwise
Inequality	!=	x != y	true if x does not equal y, false otherwise

You have already seen how most of these work, and they are pretty intuitive. Each of these operators evaluates to the boolean value true (1), or false (0).

Here's some sample code using these operators with integers:

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";
    int x{};
    std::cin >> x;

    std::cout << "Enter another integer: ";
    int y{};
    std::cin >> y;

    if (x == y)
        std::cout << x << " equals " << y << '\n';
    if (x != y)
        std::cout << x << " does not equal " << y << '\n';
    if (x > y)
        std::cout << x << " is greater than " << y << '\n';
    if (x < y)
        std::cout << x << " is less than " << y << '\n';
    if (x >= y)
        std::cout << x << " is greater than or equal to " << y << '\n';
    if (x <= y)
        std::cout << x << " is less than or equal to " << y << '\n';

    return 0;
}
```

And the results from a sample run:

```
Enter an integer: 4
Enter another integer: 5
4 does not equal 5
4 is less than 5
4 is less than or equal to 5
```

These operators are extremely straightforward to use when comparing integers.

Boolean conditional values

By default, conditions in an *if statement* or *conditional operator* (and a few other places) evaluate as Boolean values.

Many new programmers will write statements like this one:

```
if (b1 == true) ...
```

This is redundant, as the `== true` doesn't actually add any value to the condition. Instead, we should write:

```
if (b1) ...
```

Similarly, the following:

```
if (b1 == false) ...
```

is better written as:

```
if (!b1) ...
```

> **Best practice**
>
> Don't add unnecessary == or != to conditions. It makes them harder to read without offering any additional value.

Comparison of calculated floating point values can be problematic

Consider the following program:

```
#include <iostream>

int main()
{
    constexpr double d1{ 100.0 - 99.99 }; // should equal 0.01 mathematically
    constexpr double d2{ 10.0 - 9.99 }; // should equal 0.01 mathematically

    if (d1 == d2)
        std::cout << "d1 == d2" << '\n';
    else if (d1 > d2)
        std::cout << "d1 > d2" << '\n';
    else if (d1 < d2)
        std::cout << "d1 < d2" << '\n';

    return 0;
}
```

Variables d1 and d2 should both have value *0.01*. But this program prints an unexpected result:

```
d1 > d2
```

If you inspect the value of d1 and d2 in a debugger, you'd likely see that d1 = 0.010000000000005116 and d2 = 0.0099999999999997868. Both numbers are close to 0.01, but d1 is greater than, and d2 is less than.

Comparing floating point values using any of the relational operators can be dangerous. This is because floating point values are not precise, and small rounding errors in the floating point operands may cause them to be slightly smaller or slightly larger than expected. And this can throw off the relational operators.

> **Related content**
>
> We discussed rounding errors in lesson 4.8 – Floating point numbers.

Floating point less-than and greater-than

When the less-than (`<`), greater-than (`>`), less-than-equals (`<=`), and greater-than-equals (`>=`) operators are used with floating point values, they will produce a reliable answer in most cases (when the value of the operands is not similar). However, if the operands are almost identical, these operators should be considered unreliable. For example, `d1 > d2` happens to produce `true` in the above example, but could have just as easily produced `false` if the numerical errors had gone a different direction.

If the consequence of getting a wrong answer when the operands are similar is acceptable, then using these operators can be acceptable. This is an application-specific decision.

For example, consider a game (such as Space Invaders) where you want to determine whether two moving objects (such as a missile and an alien) intersect. If the objects are still far apart, these operators will return the correct answer. If the two objects are extremely close together, you might get an answer either way. In such cases, the wrong answer probably wouldn't even be noticed (it would just look like a near miss, or near hit) and the game would continue.

Floating point equality and inequality

The equality operators (`==` and `!=`) are much more troublesome. Consider operator==, which returns true only if its operands are exactly equal. Because even the smallest rounding error will cause two floating point numbers to not be equal, operator== is at high risk for returning false when a true might be expected. Operator!= has the same kind of problem.

```
#include <iostream>

int main()
{
    std::cout << std::boolalpha << (0.3 == 0.2 + 0.1); // prints false

    return 0;
}
```

For this reason, use of these operators with floating point operands should generally be avoided.

> **Warning**
>
> Avoid using operator== and operator!= to compare floating point values if there is any chance those values have been calculated.

There is one notable exception case to the above: It is safe to compare a floating point literal with a variable of the same type that has been initialized with a literal of the same type, so long as the number of significant digits in each literal does not exceed the minimum precision for that type. Float has a minimum precision of 6 significant digits, and double has a minimum precision of 15 significant digits.

We cover the precision for the different types in lesson 4.8 – Floating point numbers.

For example, you may occasionally see a function that returns a floating point literal (typically `0.0`, or sometimes `1.0`). In such cases, it is safe to do a direct comparison against the same literal value of the same type:

```
if (someFcn() == 0.0) // okay if someFcn() returns 0.0 as a literal only
    // do something
```

Instead of a literal, we can also compare a const or constexpr floating point variable that was initialized with a literal value:

```
constexpr double gravity { 9.8 };
if (gravity == 9.8) // okay if gravity was initialized with a literal
    // we're on earth
```

It is mostly not safe to compare floating point literals of different types. For example, comparing `9.8f` to `9.8` will return false.

> **Tip**
>
> It is safe to compare a floating point literal with a variable of the same type that has been initialized with a literal of the same type, so long as the number of significant digits in each literal does not exceed the minimum precision for that type. Float has a minimum precision of 6 significant digits, and double has a minimum precision of 15 significant digits.
>
> It is generally not safe to compare floating point literals of different types.

Comparing floating point numbers (advanced / optional reading)

So how can we reasonably compare two floating point operands to see if they are equal?

The most common method of doing floating point equality involves using a function that looks to see if two numbers are *almost* the same. If they are "close enough", then we call them equal. The value used to represent "close enough" is traditionally called **epsilon**. Epsilon is generally defined as a small positive number (e.g. 0.00000001, sometimes written 1e-8).

New developers often try to write their own "close enough" function like this:

```
#include <cmath> // for std::abs()

// absEpsilon is an absolute value
bool approximatelyEqualAbs(double a, double b, double absEpsilon)
{
    // if the distance between a and b is less than or equal to absEpsilon, then a and b are "close enough"
    return std::abs(a - b) <= absEpsilon;
}
```

std::abs() is a function in the `<cmath>` header that returns the absolute value of its argument. So `std::abs(a - b) <= absEpsilon` checks if the distance between *a* and *b* is less than or equal to whatever epsilon value representing "close enough" was passed in. If *a* and *b* are close enough, the function returns true to indicate they're equal. Otherwise, it returns false.

While this function can work, it's not great. An epsilon of *0.00001* is good for inputs around *1.0*, too big for inputs around *0.0000001*, and too small for inputs like *10,000*.

> **As an aside…**
>
> If we say any number that is within 0.00001 of another number should be treated as the same number, then:
>
> - 1 and 1.0001 would be different, but 1 and 1.00001 would be the same. That's not unreasonable.
> - 0.0000001 and 0.00001 would be the same. That doesn't seem good, as those numbers are two orders of magnitude apart.
> - 10000 and 10000.0001 would be different. That also doesn't seem good, as those numbers are barely different given the magnitude of the number.

This means every time we call this function, we have to pick an epsilon that's appropriate for our inputs. If we know we're going to have to scale epsilon in proportion to the magnitude of our inputs, we might as well modify the function to do that for us.

Donald Knuth, a famous computer scientist, suggested the following method in his book "The Art of Computer Programming, Volume II: Seminumerical Algorithms (Addison-Wesley, 1969)":

```
#include <algorithm> // for std::max
#include <cmath>     // for std::abs

// Return true if the difference between a and b is within epsilon percent of the larger of a and b
bool approximatelyEqualRel(double a, double b, double relEpsilon)
{
    return (std::abs(a - b) <= (std::max(std::abs(a), std::abs(b)) * relEpsilon));
}
```

In this case, instead of epsilon being an absolute number, epsilon is now relative to the magnitude of *a* or *b*.

Let's examine in more detail how this crazy looking function works. On the left side of the <= operator, `std::abs(a - b)` tells us the distance between *a* and *b* as a positive number.

On the right side of the <= operator, we need to calculate the largest value of "close enough" we're willing to accept. To do this, the algorithm chooses the larger of *a* and *b* (as a rough indicator of the overall magnitude of the numbers), and then multiplies it by relEpsilon. In this function, relEpsilon represents a percentage. For example, if we want to say "close enough" means *a* and *b* are within 1% of the larger of *a* and *b*, we pass in an relEpsilon of 0.01 (1% = 1/100 = 0.01). The value for relEpsilon can be adjusted to whatever is most appropriate for the circumstances (e.g. an epsilon of 0.002 means within 0.2%).

To do inequality (!=) instead of equality, simply call this function and use the logical NOT operator (!) to flip the result:

```
if (!approximatelyEqualRel(a, b, 0.001))
    std::cout << a << " is not equal to " << b << '\n';
```

Note that while the approximatelyEqualRel() function will work for most cases, it is not perfect, especially as the numbers approach zero:

```
#include <algorithm> // for std::max
#include <cmath>     // for std::abs
#include <iostream>

// Return true if the difference between a and b is within epsilon percent of the larger of a and b
bool approximatelyEqualRel(double a, double b, double relEpsilon)
{
    return (std::abs(a - b) <= (std::max(std::abs(a), std::abs(b)) * relEpsilon));
}

int main()
{
    // a is really close to 1.0, but has rounding errors
    constexpr double a{ 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 };

    constexpr double relEps { 1e-8 };
    constexpr double absEps { 1e-12 };

    std::cout << std::boolalpha; // print true or false instead of 1 or 0

    // First, let's compare a (almost 1.0) to 1.0.
    std::cout << approximatelyEqualRel(a, 1.0, relEps) << '\n';

    // Second, let's compare a-1.0 (almost 0.0) to 0.0
    std::cout << approximatelyEqualRel(a-1.0, 0.0, relEps) << '\n';

    return 0;
}
```

Perhaps surprisingly, this returns:

```
true
false
```

The second call didn't perform as expected. The math simply breaks down close to zero.

One way to avoid this is to use both an absolute epsilon (as we did in the first approach) and a relative epsilon (as we did in Knuth's approach):

```
// Return true if the difference between a and b is less than or equal to absEpsilon, or within relEpsilon percent o
larger of a and b
bool approximatelyEqualAbsRel(double a, double b, double absEpsilon, double relEpsilon)
{
    // Check if the numbers are really close -- needed when comparing numbers near zero.
    if (std::abs(a - b) <= absEpsilon)
        return true;

    // Otherwise fall back to Knuth's algorithm
    return approximatelyEqualRel(a, b, relEpsilon);
}
```

In this algorithm, we first check if *a* and *b* are close together in absolute terms, which handles the case where *a* and *b* are both close to zero. The *absEpsilon* parameter should be set to something very small (e.g. 1e-12). If that fails, then we fall back to Knuth's algorithm, using the relative epsilon.

Here's our previous code testing both algorithms:

```
#include <algorithm> // for std::max
#include <cmath>     // for std::abs
#include <iostream>

// Return true if the difference between a and b is within epsilon percent of the larger of a and b
bool approximatelyEqualRel(double a, double b, double relEpsilon)
{
    return (std::abs(a - b) <= (std::max(std::abs(a), std::abs(b)) * relEpsilon));
}

// Return true if the difference between a and b is less than or equal to absEpsilon, or within relEpsilon percent o
larger of a and b
bool approximatelyEqualAbsRel(double a, double b, double absEpsilon, double relEpsilon)
{
    // Check if the numbers are really close -- needed when comparing numbers near zero.
    if (std::abs(a - b) <= absEpsilon)
        return true;

    // Otherwise fall back to Knuth's

---

# 6.8 — Logical operators

While relational (comparison) operators can be used to test whether a particular condition is true or false, they ca
test one condition at a time. Often we need to know whether multiple conditions are true simultaneously. For example, to
check whether we've won the lottery, we have to compare whether all of the multiple numbers we picked match the winning
numbers. In a lottery with 6 numbers, this would involve 6 comparisons, *all* of which have to be true. In other cases, we
need to know whether any one of multiple conditions is true. For example, we may decide to skip work today if we're sick,
or if we're too tired, or if we won the lottery in our previous example. This would involve checking whether *any* of 3
comparisons is true.

Logical operators provide us with the capability to test multiple conditions.

C++ has 3 logical operators:

| Operator | Symbol | Example Usage | Operation |
| --- | --- | --- | --- |
| Logical NOT | ! | !x | true if x is false, or false if x is true |
| Logical AND | && | x && y | true if x and y are both true, false otherwise |
| Logical OR | \|\| | x \|\| y | true if either (or both) x or y are true, false otherwise |

## Logical NOT

You have already run across the logical NOT unary operator in lesson 4.9 -- Boolean values. We can summarize the eff
of logical NOT like so:

| Logical NOT (operator !) | |
```

```markdown
| --- | --- |
| Operand | Result |
| true | false |
| false | true |

If *logical NOT's* operand evaluates to true, *logical NOT* evaluates to false. If *logical NOT's* operand evaluates false, *logical NOT* evaluates to true. In other words, *logical NOT* flips a Boolean value from true to false, and vice-versa.

Logical NOT is often used in conditionals:

```cpp
bool tooLarge { x > 100 }; // tooLarge is true if x > 100
if (!tooLarge)
    // do something with x
else
    // print an error
```
```

One thing to be wary of is that *logical NOT* has a very high level of precedence. New programmers often make the following mistake:

```cpp
#include <iostream>

int main()
{
    int x{ 5 };
    int y{ 7 };

    if (!x > y)
        std::cout << x << " is not greater than " << y << '\n';
    else
        std::cout << x << " is greater than " << y << '\n';

    return 0;
}
```

This program prints:

```
5 is greater than 7
```

But *x* is not greater than *y*, so how is this possible? The answer is that because the *logical NOT* operator has higher precedence than the *greater than* operator, the expression `! x > y` actually evaluates as `(!x) > y`. Since *x* is 5, !x evaluates to *0*, and `0 > y` is false, so the *else* statement executes!

The correct way to write the above snippet is:

```
#include <iostream>

int main()
{
    int x{ 5 };
    int y{ 7 };

    if (!(x > y))
        std::cout << x << " is not greater than " << y << '\n';
    else
        std::cout << x << " is greater than " << y << '\n';

    return 0;
}
```

This way, `x > y` will be evaluated first, and then logical NOT will flip the Boolean result.

> **Best practice**
>
> If *logical NOT* is intended to operate on the result of other operators, the other operators and their operands need to be enclosed in parentheses.

Simple uses of *logical NOT*, such as `if (!value)` do not need parentheses because precedence does not come into play.

Logical OR

The *logical OR* operator is used to test whether either of two conditions is true. If the left operand evaluates to true, or the right operand evaluates to true, or both are true, then the *logical OR* operator returns true. Otherwise it will return false.

Logical OR (operator ||)		
Left operand	Right operand	Result
false	false	false
false	true	true
true	false	true
true	true	true

For example, consider the following program:

```
#include <iostream>

int main()
{
    std::cout << "Enter a number: ";
    int value {};
    std::cin >> value;

    if (value == 0 || value == 1)
        std::cout << "You picked 0 or 1\n";
    else
        std::cout << "You did not pick 0 or 1\n";
    return 0;
}
```

In this case, we use the logical OR operator to test whether either the left condition (value == 0) or the right condition (value == 1) is true. If either (or both) are true, the *logical OR* operator evaluates to true, which means the *if statement* executes. If neither are true, the *logical OR* operator evaluates to false, which means the *else statement* executes.

> **Warning**
>
> New programmers sometimes try this:
>
> ```
> if (value == 0 || 1) // incorrect: if value is 0, or if 1
> ```
>
> When `1` is evaluated, it will implicitly convert to `bool` `true`. Thus this conditional will always evaluate to `true`.
>
> If you want to compare a variable against multiple values, you need to compare the variable multiple times:
>
> ```
> if (value == 0 || value == 1) // correct: if value is 0, or if value is 1
> ```

You can string together many *logical OR* statements:

```
if (value == 0 || value == 1 || value == 2 || value == 3)
    std::cout << "You picked 0, 1, 2, or 3\n";
```

New programmers sometimes confuse the *logical OR* operator (||) with the *bitwise OR* operator (|) (covered in lesson O.2 – Bitwise operators). Even though they both have *OR* in the name, they perform different functions. Mixing them up will probably lead to incorrect results.

Logical AND

The *logical AND* operator is used to test whether both operands are true. If both operands are true, *logical AND* returns true. Otherwise, it returns false.

Logical AND (operator &&)		
Left operand	Right operand	Result
false	false	false
false	true	false
true	false	false
true	true	true

For example, we might want to know if the value of variable *x* is between *10* and *20*. This is actually two conditions: we need to know if *x* is greater than *10*, and also whether *x* is less than *20*.

```
#include <iostream>

int main()
{
    std::cout << "Enter a number: ";
    int value {};
    std::cin >> value;

    if (value > 10 && value < 20)
        std::cout << "Your value is between 10 and 20\n";
    else
        std::cout << "Your value is not between 10 and 20\n";
    return 0;
}
```

In this case, we use the *logical AND* operator to test whether the left condition (value > 10) AND the right condition (value < 20) are both true. If both are true, the *logical AND* operator evaluates to true, and the *if statement* executes. If neither are true, or only one is true, the *logical AND* operator evaluates to false, and the *else statement* executes.

As with *logical OR*, you can string together many *logical AND* statements:

```
if (value > 10 && value < 20 && value != 16)
    // do something
else
    // do something else
```

If all of these conditions are true, the *if statement* will execute. If any of these conditions are false, the *else statement* will execute.

As with logical and bitwise OR, new programmers sometimes confuse the *logical AND* operator (&&) with the *bitwise AND* operator (&).

Short circuit evaluation

In order for *logical AND* to return true, both operands must evaluate to true. If the left operand evaluates to false, *logical AND* knows it must return false regardless of whether the right operand evaluates to true or false. In this case, the *logical AND* operator will go ahead and return false immediately without even evaluating the right operand! This is known as **short circuit evaluation**, and it is done primarily for optimization purposes.

Similarly, if the left operand for *logical OR* is true, then the entire OR condition has to evaluate to true, and the right operand won't be evaluated.

Short circuit evaluation presents another opportunity to show why operators that cause side effects should not be used in compound expressions. Consider the following snippet:

```
if (x == 1 && ++y == 2)
    // do something
```

if *x* does not equal *1*, the whole condition must be false, so ++y never gets evaluated! Thus, *y* will only be incremented if *x* evaluates to 1, which is probably not what the programmer intended!

> **Warning**
>
> Short circuit evaluation may cause *Logical OR* and *Logical AND* to not evaluate the right operand. Avoid using expressions with side effects in conjunction with these operators.

> **Key insight**
>
> The Logical OR and logical AND operators are an exception to the rule that the operands may evaluate in any order, as the standard explicitly states that the left operand must evaluate first.

> **For advanced readers**
>
> Only the built-in versions of these operators perform short-circuit evaluation. If you overload these operators to make them work with your own types, those overloaded operators will not perform short-circuit evaluation.

Mixing ANDs and ORs

Mixing *logical AND* and *logical OR* operators in the same expression often can not be avoided, but it is an area full of potential dangers.

Because *logical AND* and *logical OR* seem like a pair, many programmers assume they have the same precedence (just like addition/subtraction and multiplication/division). However, *logical AND* has higher precedence than *logical OR*, thus *logical AND* operators will be evaluated ahead of *logical OR* operators (unless they have been parenthesized).

New programmers will often write expressions such as `value1 || value2 && value3`. Because *logical AND* has higher precedence, this evaluates as `value1 || (value2 && value3)`, not `(value1 || value2) && value3`. Hopefully that's what the programmer wanted! If the programmer was assuming left to right association (as happens with addition/subtraction, or multiplication/division), the programmer will get a result he or she was not expecting!

When mixing *logical AND* and *logical OR* in the same expression, it is a good idea to explicitly parenthesize each operator and its operands. This helps prevent precedence mistakes, makes your code easier to read, and clearly defines how you intended the expression to evaluate. For example, rather than writing `value1 && value2 || value3 && value4`, it is better to write `(value1 && value2) || (value3 && value4)`.

> **Best practice**
>
> When mixing *logical AND* and *logical OR* in a single expression, explicitly parenthesize each operation to ensure they evaluate how you intend.

De Morgan's laws

Many programmers also make the mistake of thinking that `!(x && y)` is the same thing as `!x && !y`. Unfortunately, you can not "distribute" the *logical NOT* in that manner.

De Morgan's laws tell us how the *logical NOT* should be distributed in these cases:

`!(x && y)` is equivalent to `!x || !y` `!(x || y)` is equivalent to `!x && !y`

In other words, when you distribute the *logical NOT*, you also need to flip *logical AND* to *logical OR*, and vice-versa!

This can sometimes be useful when trying to make complex expressions easier to read.

> **For advanced readers**

> We can show that the first part of De Morgan's Laws is correct by proving that `!(x && y)` equals `!x || !y` for every possible value of `x` and `y`. To do so, we'll use a mathematical concept called a truth table:
>
> | x | y | !x | !y | !(x && y) | !x \|\| !y |
> |---|---|---|---|---|---|
> | false | false | true | true | true | true |
> | false | true | true | false | true | true |
> | true | false | false | true | true | true |
> | true | true | false | false | false | false |
>
> In this table, the first and second columns represent our `x` and `y` variables. Each row in the table shows one permutation of possible values for `x` and `y`. Because `x` and `y` are Boolean values, we only need 4 rows to cover every combination of possible values that `x` and `y` can hold.
>
> The rest of the columns in the table represent expressions that we want to evaluate based on the initial values of `x` and `y`. The third and fourth columns calculate the values of `!x` and `!y` respectively. The fifth column calculates the value of `!(x && y)`. Finally, the sixth column calculates the value of `!x || !y`.
>
> You'll notice for each row, the value in the fifth column matches the value in the sixth column. This means for every possible value of `x` and `y`, the value of `!(x && y)` equals `!x || !y`, which is what we were trying to prove!
>
> We can do the same for the second part of De Morgan's Laws:
>
> | x | y | !x | !y | !(x \|\| y) | !x && !y |
> |---|---|---|---|---|---|
> | false | false | true | true | true | true |
> | false | true | true | false | false | false |
> | true | false | false | true | false | false |
> | true | true | false | false | false | false |
>
> Similarly, for every possible value of `x` and `y`, we can see that the value of `!(x || y)` equals the value of `!x && !y`. Thus, they are equivalent.

Where's the logical exclusive or (XOR) operator?

Logical XOR is a logical operator provided in some languages that is used to test whether an odd number of conditions is true:

Logical XOR		
Left operand	Right operand	Result
false	false	false
false	true	true
true	false	true
true	true	false

C++ doesn't provide an explicit *logical XOR* operator (`operator^` is a bitwise XOR, not a logical XOR). Unlike *logical OR* or *logical AND*, *logical XOR* cannot be short circuit evaluated. Because of this, making a *logical XOR* operator out of *logical OR* and *logical AND* operators is challenging.

However, `operator!=` produces the same result as a logical XOR when given `bool` operands:

Left operand	Right operand	logical XOR	operator!=
false	false	false	false
false	true	true	true
true	false	true	true
true	true	false	false

Therefore, a logical XOR can be implemented as follows:

```
if (a != b) ... // a XOR b, assuming a and b are bool
```

This can be extended to multiple operands as follows:

```
if (a != b != c) ... // a XOR b XOR c, assuming a, b, and c are bool
```

This evaluates to `true` if an odd number of the operands (`a` , `b` , and `c`) evaluate to `true` .

If the operands are not of type `bool` , using `operator!=` to implement a logical XOR will not work as expected.

> **For advanced readers**
>
> If you need a form of *logical XOR* that works with non-Boolean operands, you can static_cast your operands to bool:
>
> ```
> if (static_cast<bool>(a) != static_cast<bool>(b) != static_cast<bool>(c)) ... // a XOR b XOR c, for any type that be converted to bool
> ```

6.x — Chapter 6 summary and quiz

Quick review

Always use parentheses to disambiguate the precedence of operators if there is any question or opportunity for confusion.

The arithmetic operators all work like they do in normal mathematics. The remainder (`%`) operator returns the remainder from an integer division.

The increment and decrement operators can be used to easily increment or decrement numbers. Avoid the postfix versions of these operators whenever possible.

Beware of side effects, particularly when it comes to the order that function parameters are evaluated. Do not use a variable that has a side effect applied more than once in a given statement.

The comma operator can compress multiple statements into one. Writing the statements separately is usually better.

The **conditional operator** (`?:`) (also sometimes called the **arithmetic if** operator) is a ternary operator (an operator that takes 3 operands). Given a conditional operation of the form `c ? x : y` , if conditional `c` evaluates to `true` then `x` will be evaluated, otherwise `y` will be evaluated. The conditional operator typically needs to be parenthesized as follows:

- Parenthesize the entire conditional operator when used in a compound expression (an expression with other operators).
- For readability, parenthesize the condition if it contains any operators (other than the function call operator).

Relational operators can be used to compare floating point numbers. Beware using equality and inequality on floating point numbers.

Logical operators allow us to form compound conditional statements.

Quiz time

Complete the following program:

```
#include <iostream>

// Write the function getQuantityPhrase() here

// Write the function getApplesPluralized() here

int main()
{
    constexpr int maryApples { 3 };
    std::cout << "Mary has " << getQuantityPhrase(maryApples) << ' ' << getApplesPluralized(maryApples) << ".\n";

    std::cout << "How many apples do you have? ";
    int numApples{};
    std::cin >> numApples;

    std::cout << "You have " << getQuantityPhrase(numApples) << ' ' << getApplesPluralized(numApples) << ".\n";

    return 0;
}
```

Sample output:

```
Mary has a few apples.
How many apples do you have? 1
You have a single apple.
```

`getQuantityPhrase()` should take a single int parameter representing the quantity of something and return the following descriptor:

- < 0 = “negative”
- 0 = “no”
- 1 = “a single”
- 2 = “a couple of”
- 3 = “a few”
- > 3 = “many”

`getApplesPluralized()` should take a single int parameter parameter representing the quantity of apples and return the following:

- 1 = “apple”
- otherwise = “apples”

This function should use the conditional operator.

> **Hint:** It’s okay to return a C-style string literal from a function as a `std::string_view`.

```cpp
#include <iostream>
#include <string_view>

std::string_view getQuantityPhrase(int num)
{
    if (num < 0)
        return "negative";
    if (num == 0)
        return "no";
    if (num == 1)
        return "a single";
    if (num == 2)
        return "a couple of";
    if (num == 3)
        return "a few";
    return "many";
}

std::string_view getApplesPluralized(int num)
{
    return (num == 1) ? "apple" : "apples";
}

int main()
{
    constexpr int maryApples { 3 };
    std::cout << "Mary has " << getQuantityPhrase(maryApples) << ' ' << getApplesPluralized(maryApples) << ".\n";

    std::cout << "How many apples do you have? ";
    int numApples{};
    std::cin >> numApples;

    std::cout << "You have " << getQuantityPhrase(numApples) << ' ' << getApplesPluralized(numApples) << ".\n";

    return 0;
}
```